

Name: Abhinav Garg

ID: 2023ucp1706

Ans.1.

Without Barrier,

Code:

```
#include <stdio.h>
#include <omp.h>

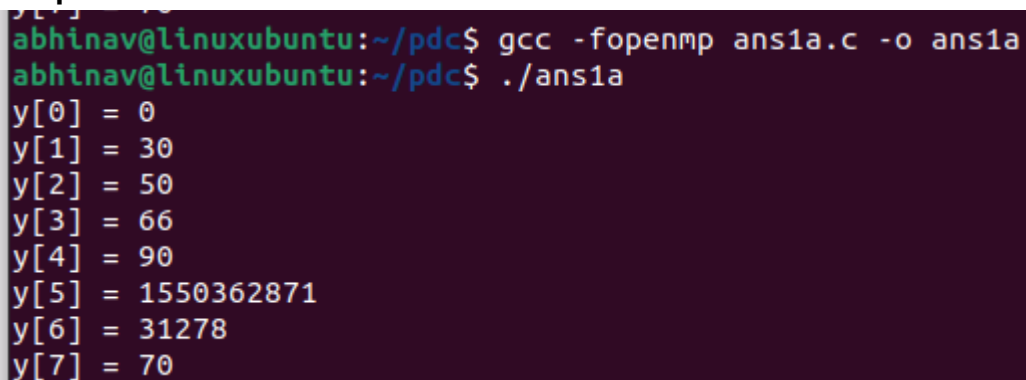
#define N 8

int main() {
    int x[N], y[N];

    #pragma omp parallel num_threads(N)
    {
        int mytid = omp_get_thread_num();
        x[mytid] = mytid * 10;
        y[mytid] = x[mytid] + x[mytid + 1];
    }

    for (int i = 0; i < N; i++) {
        printf("y[%d] = %d\n", i, y[i]);
    }
    return 0;
}
```

Output:



```
abhinav@linuxubuntu:~/pdc$ gcc -fopenmp ans1a.c -o ans1a
abhinav@linuxubuntu:~/pdc$ ./ans1a
y[0] = 0
y[1] = 30
y[2] = 50
y[3] = 66
y[4] = 90
y[5] = 1550362871
y[6] = 31278
y[7] = 70
```

With Barrier,

Code:

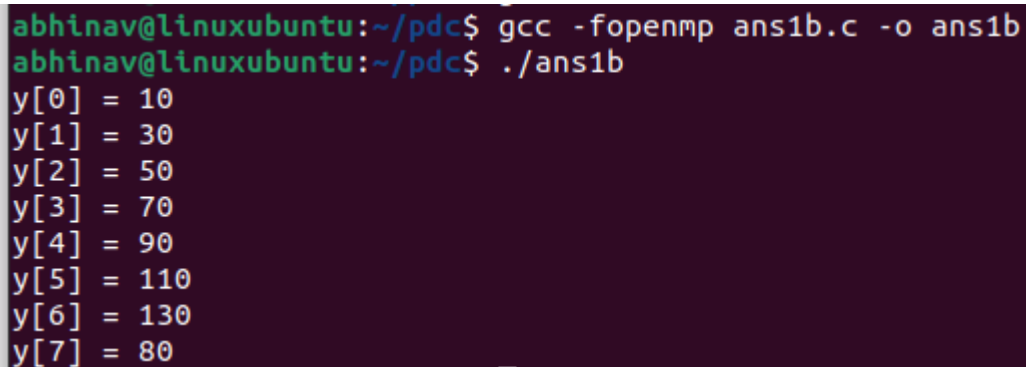
```
#include <stdio.h>
#include <omp.h>

#define N 8

int main() {
    int x[N], y[N];

    #pragma omp parallel num_threads(N)
    {
        int mytid = omp_get_thread_num();
        x[mytid] = mytid * 10;
        #pragma omp barrier
        y[mytid] = x[mytid] + x[mytid + 1];
    }

    for (int i = 0; i < N; i++) {
        printf("y[%d] = %d\n", i, y[i]);
    }
    return 0;
}
```

Output:

```
abhinav@linuxubuntu:~/pdc$ gcc -fopenmp ans1b.c -o ans1b
abhinav@linuxubuntu:~/pdc$ ./ans1b
y[0] = 10
y[1] = 30
y[2] = 50
y[3] = 70
y[4] = 90
y[5] = 110
y[6] = 130
y[7] = 80
```

Ans.2. (Implicit Barrier)

```
#include <stdio.h>
#include <omp.h>
#define N 10

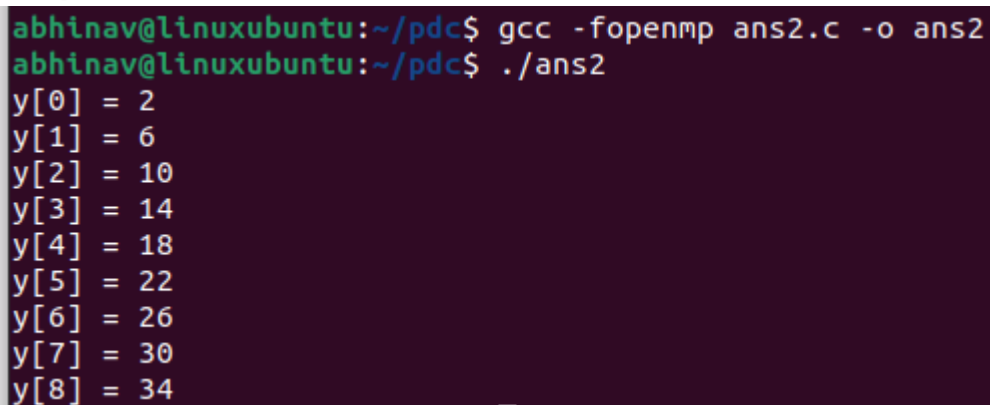
int main() {
    int x[N], y[N - 1];

    #pragma omp parallel num_threads(4)
    {
        #pragma omp for
        for (int i = 0; i < N; i++)
            x[i] = i * 2;

        #pragma omp for
        for (int i = 0; i < N - 1; i++)
            y[i] = x[i] + x[i + 1];
    }

    for (int i = 0; i < N - 1; i++) {
        printf("y[%d] = %d\n", i, y[i]);
    }
    return 0;
}
```

Output:



```
abhinav@linuxubuntu:~/pdc$ gcc -fopenmp ans2.c -o ans2
abhinav@linuxubuntu:~/pdc$ ./ans2
y[0] = 2
y[1] = 6
y[2] = 10
y[3] = 14
y[4] = 18
y[5] = 22
y[6] = 26
y[7] = 30
y[8] = 34
```

Ans.3. (nowait)

Code:

```
#include <stdio.h>
#include <omp.h>

#define N 10

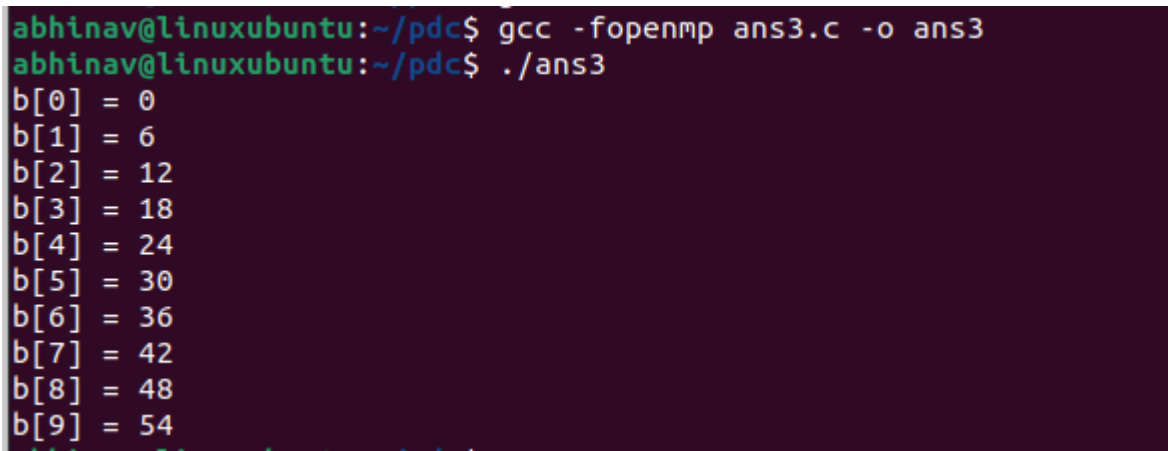
int main() {
    int a[N], b[N];

    #pragma omp parallel num_threads(4)
    {
        #pragma omp for nowait
        for (int i = 0; i < N; i++)
            a[i] = i * 3;

        #pragma omp for
        for (int i = 0; i < N; i++)
            b[i] = a[i] * 2;
    }

    for (int i = 0; i < N; i++) {
        printf("b[%d] = %d\n", i, b[i]);
    }
    return 0;
}
```

Output:



```
abhinav@linuxubuntu:~/pdc$ gcc -fopenmp ans3.c -o ans3
abhinav@linuxubuntu:~/pdc$ ./ans3
b[0] = 0
b[1] = 6
b[2] = 12
b[3] = 18
b[4] = 24
b[5] = 30
b[6] = 36
b[7] = 42
b[8] = 48
b[9] = 54
```

Ans.4. (Mutual exclusion)

Code:

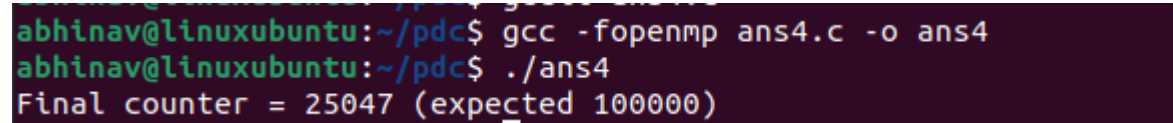
```
#include <stdio.h>
#include <omp.h>

int main() {
    int counter = 0;
    const int count = 100000;

    #pragma omp parallel for shared(counter) num_threads(4)
    for (int i = 0; i < count; i++)
        counter += 1;

    printf("Final counter = %d (expected %d)\n", counter, count);
    return 0;
}
```

Output:



```
abhinav@linuxubuntu:~/pdc$ gcc -fopenmp ans4.c -o ans4
abhinav@linuxubuntu:~/pdc$ ./ans4
Final counter = 25047 (expected 100000)
```

Ans.5. (locks)

Code:

```
#include <stdio.h>
#include <omp.h>

int main() {
    omp_lock_t mylock;
    omp_init_lock(&mylock);

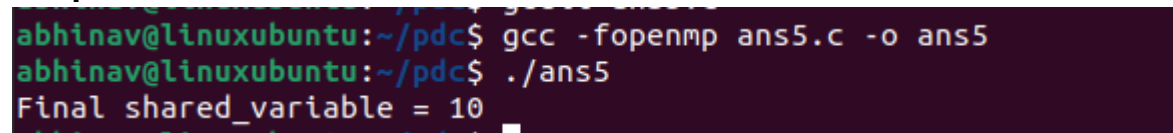
    int shared_variable = 0;

    #pragma omp parallel num_threads(4) shared(shared_variable)
    {
        omp_set_lock(&mylock);
        shared_variable += omp_get_thread_num() + 1;
        omp_unset_lock(&mylock);
    }

    omp_destroy_lock(&mylock);

    printf("Final shared_variable = %d\n", shared_variable);
    return 0;
}
```

Output:



```
abhinav@linuxubuntu:~/pdc$ gcc -fopenmp ans5.c -o ans5
abhinav@linuxubuntu:~/pdc$ ./ans5
Final shared_variable = 10
```

Ans.6. (Deadlocks)

Code:

```
#include <stdio.h>
#include <omp.h>

#define SIZE 5

int main() {
    int a[SIZE], b[SIZE];
    omp_lock_t locka, lockb;
    omp_init_lock(&locka);
    omp_init_lock(&lockb);

    #pragma omp parallel shared(a, b, locka, lockb) num_threads(2)
    #pragma omp sections nowait
    {
        #pragma omp section
        {
            omp_set_lock(&locka);
            for (int i = 0; i < SIZE; i++) a[i] = i;
            omp_set_lock(&lockb);
            for (int i = 0; i < SIZE; i++) b[i] = a[i] * 2;
            omp_unset_lock(&lockb);
            omp_unset_lock(&locka);
        }

        #pragma omp section
        {
            omp_set_lock(&lockb);
            for (int i = 0; i < SIZE; i++) b[i] = i * 3;
            omp_set_lock(&locka);
            for (int i = 0; i < SIZE; i++) a[i] = b[i] / 2;
            omp_unset_lock(&locka);
            omp_unset_lock(&lockb);
        }
    }

    omp_destroy_lock(&locka);
    omp_destroy_lock(&lockb);
}
```

```
    printf("If it didn't deadlock: a[0]=%d, b[0]=%d\n", a[0], b[0]);  
    return 0;  
}
```

Output:

```
abhinav@linuxubuntu:~/pdc$ gcc -fopenmp ans6.c -o ans6  
abhinav@linuxubuntu:~/pdc$ ./ans6  
If it didn't deadlock: a[0]=0, b[0]=0
```

Ans.7. (Deadlock in same order)

Code:

```
#include <stdio.h>  
#include <omp.h>  
  
#define SIZE 5  
  
int main() {  
    int a[SIZE], b[SIZE];  
    omp_lock_t locka, lockb;  
    omp_init_lock(&locka);  
    omp_init_lock(&lockb);  
  
    #pragma omp parallel shared(a, b, locka, lockb) num_threads(2)  
    #pragma omp sections nowait  
    {  
        #pragma omp section  
        {  
            omp_set_lock(&locka);  
            omp_set_lock(&lockb);  
            for (int i = 0; i < SIZE; i++) {  
                a[i] = i;  
                b[i] = a[i] * 2;  
            }  
            omp_unset_lock(&lockb);  
            omp_unset_lock(&locka);  
        }  
  
        #pragma omp section  
        {
```



```

    omp_set_lock(&locka);
    omp_set_lock(&lockb);
    for (int i = 0; i < SIZE; i++) {
        b[i] = i * 3;
        a[i] = b[i] / 2;
    }
    omp_unset_lock(&lockb);
    omp_unset_lock(&locka);
}
}

omp_destroy_lock(&locka);
omp_destroy_lock(&lockb);

printf("a[0]=%d, b[0]=%d\n", a[0], b[0]);
return 0;
}

```

Output:

```

abhinav@linuxubuntu:~/pdc$ gcc -fopenmp ans7.c -o ans7
abhinav@linuxubuntu:~/pdc$ ./ans7
a[0]=0, b[0]=0

```

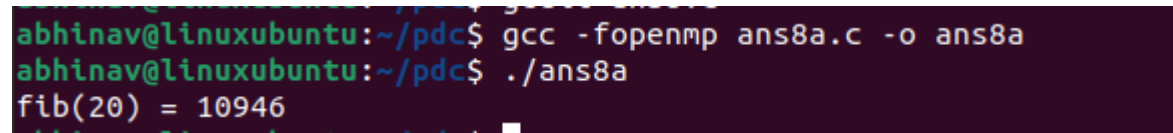
Ans.8. (task for recursive problems)

Simple Fibonacci,

Code:

```
#include <stdio.h>
int fib(int n) {
    if (n < 2) return 1;
    return fib(n - 1) + fib(n - 2);
}
int main() {
    printf("fib(20) = %d\n", fib(20));
    return 0;
}
```

Output:



```
abhinav@linuxubuntu:~/pdc$ gcc -fopenmp ans8a.c -o ans8a
abhinav@linuxubuntu:~/pdc$ ./ans8a
fib(20) = 10946
```

Optimized Fibonacci,

Code:

```
#include <stdio.h>
#include <omp.h>
#include <stdlib.h>

#define MAXN 40

int value[MAXN];
int done[MAXN];
omp_lock_t dlock[MAXN];

int fib(int n) {
    if (n < 2) return 1;

    omp_set_lock(&dlock[n]);
    if (!done[n]) {
        int i, j;
        #pragma omp task shared(i) firstprivate(n)
        i = fib(n - 1);
        #pragma omp task shared(j) firstprivate(n)
```

```

        j = fib(n - 2);
        #pragma omp taskwait
        value[n] = i + j;
        done[n] = 1;
    }
    omp_unset_lock(&dlock[n]);
    return value[n];
}

int main() {
    for (int i = 0; i < MAXN; i++) {
        done[i] = 0;
        value[i] = 0;
        omp_init_lock(&dlock[i]);
    }

    #pragma omp parallel
    {
        #pragma omp single
        {
            printf("fib(30) = %d\n", fib(30));
        }
    }

    for (int i = 0; i < MAXN; i++)
        omp_destroy_lock(&dlock[i]);

    return 0;
}

```

Output:

```

abhinav@linuxubuntu:~/pdc$ gcc -fopenmp ans8b.c -o ans8b
abhinav@linuxubuntu:~/pdc$ ./ans8b
fib(30) = 1346269

```